

```
mirror_mod = modifier_ob.  
# Add mirror object to mirror_mod  
mirror_mod.mirror_object =  
    operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
    operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
    operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
# Selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active =  
    ("Selected" + str(modifier_ob.name))  
mirror_ob.select = 0  
bpy.context.selected_objects[0].  
data.objects[one.name].select  
print("please select exactly one mirror")
```

```
--- OPERATOR CLASSES ---  
  
class MirrorOperator(bpy.types.Operator):  
    """Mirror the selected object to the selected mirror  
    object.mirror_mirror_x"""  
    bl_label = "Mirror X"
```

```
def execute(self, context):  
    if context.active_object is not None:
```

Parallel Programming with MPI

Contents

Message Passing Systems

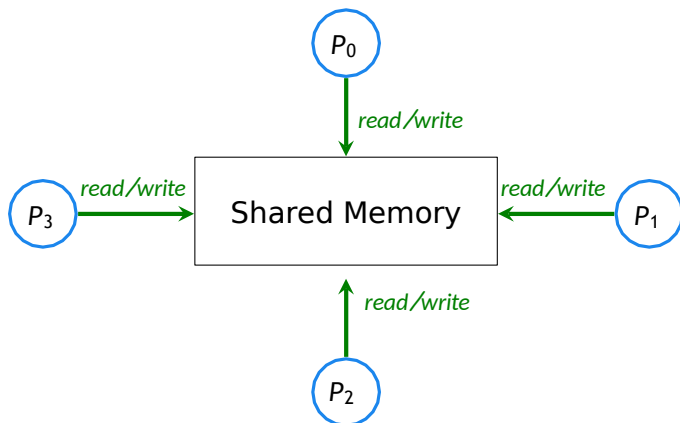
Introduction to MPI

Point-to-point
communication

Collective communication

Other features

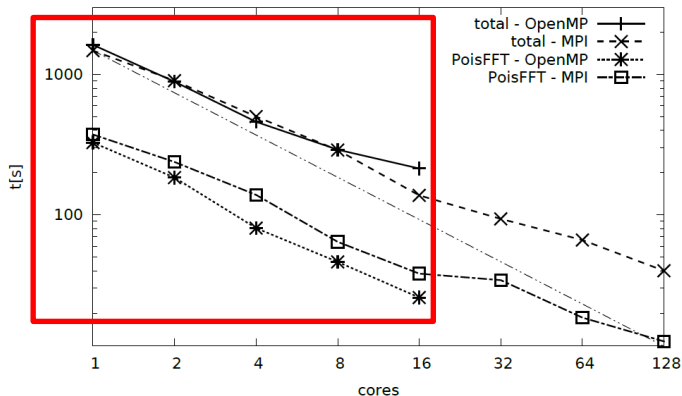
Shared memory model



- Processes have access to a shared address space
- Processes communicate by reading and writing into the shared address space
- Programming: compiler directives (OpenMP) or threads

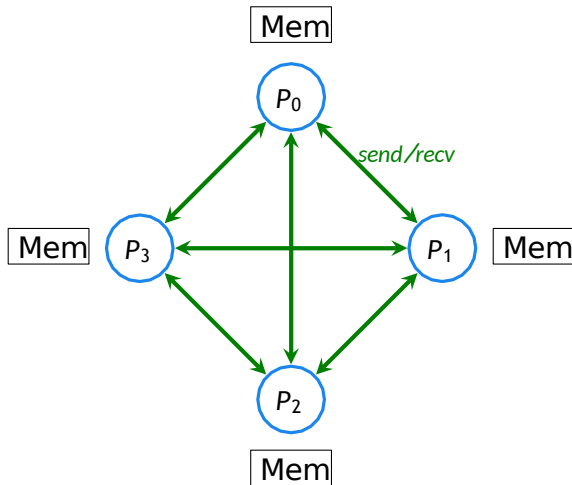
Shared memory model

- This approach scales on a single node
- It scales up to a limited numbers of cores (approx. 8)
- Used for hybrid parallelism on supercomputers
- Easy to program, less easy to make it run fast.



Distributed memory model

Message passing



- Each process has its own private memory
- Processes communicate by sending and receiving messages

Applying the models

Natural fit

- The **shared memory model** corresponds to threads executing on a single processor
- The **distributed memory model** corresponds to processes executing on servers interconnected through a network

However

- Shared memory can be implemented on top of the distributed memory model
 - ▶ Distributed shared memory
 - ▶ Partitionable Global Address Space
- The distributed memory model can be implemented on top of shared memory
 - ▶ Send/Recv operations can be implemented on top of shared memory

In a supercomputer

A large number of servers:

- Interconnected through a high-performance network
- Equipped with multicore multi-processors (and accelerators)

What programming model to use?

- Hybrid solution
 - ▶ Message passing for inter-node communication
 - ▶ Shared memory inside a node (OpenMP or MPI-3 shared memory)
- Message passing everywhere
 - ▶ Easier to program, small performance penalty (but MPI-3 could change the picture...)

Message Passing Programming Model

Differences with the shared memory model

- Communication is explicit
 - ▶ The user is in charge of managing communication
 - ▶ The programming effort is bigger
- No good automatic techniques to parallelize code
- More efficient when running on a distributed setup
 - ▶ Better control on data movements
 - ▶ Usually better also on a large number of cores in a single CPU

The Message Passing Interface (MPI)

<http://mpi-forum.org/>

MPI is the most commonly used solution to program message passing applications in the HPC context.

What's MPI?

- MPI is a standard
 - ▶ It defines a set of operations to program message passing applications.
 - ▶ The standard defines the semantic of the operations (not how they are implemented)
 - ▶ Current version is 4.1
 - ▶ Several implementations of the standard exist (libraries)
 - ▶ Open MPI and MPICH are the two main open source implementations (they provide C and Fortran bindings)

Contents

Message Passing Systems

Introduction to MPI

Point-to-point
communication

Collective communication

Other features

My first MPI program

```
#include <stdio.h> #include <string.h> #include <mpi.h>
```

```
int main(int argc, char *argv[])
{
    char msg[20]; int my_rank;
    MPI_Status status;

    MPI_Init(&argc, &argv); MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    if (my_rank == 0) {
        strcpy(msg, "Hellou!");
        MPI_Send(msg, strlen(msg), MPI_CHAR, 1, 99, MPI_COMM_WORLD);
    }
    else {
        MPI_Recv(msg, 20, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status);
        printf("I received %s!\n", msg);
    }
    MPI_Finalize();
}
```

SPMD application

MPI programs follow the Single Program Multiple Data (SPMD) execution model:

- Each process executes the same program at independent points
- Only the data differ from one process to another
- Different actions may be taken based on the rank of the process and on the data

Compiling and executing

Compiling

- Use mpicc instead of gcc (mpicxx, mpif77, mpif90)

```
mpicc -o hello_world hello_world.c
```

Executing

```
mpirun -n 2 -hostfile machine_file ./hello_world
```

- Creates 2 MPI processes that will run on the 2 first machines listed in the machine file (implementation dependent)
- If no machine file is provided, the processes are created on the local machine

Back to our example

Mandatory calls (**by all processes**)

- `MPI_Init()`: Initialize the MPI execution environment
 - ▶ No other MPI calls can be done before `Init()`.
- `MPI_Finalize()`: Terminates MPI execution environment
 - ▶ To be called before terminating the program (last executable instruction)

Note that all MPI functions are prefixed with `MPI_`

Communicators and ranks

Communicators

- A communicator defines a **group** of processes that can communicate in a communication **context**.
- Inside a communicator, processes have a unique rank
- Ranks go from 0 to $p - 1$ in a group of size p
- At the beginning of the application, a default communicator including **all** application processes is created:

MPI_COMM_WORLD

- Any communication occurs in the context of a communicator
- Processes may belong to multiple communicators and have a different rank in different communicators

Communicators and ranks: Retrieving basic info

- `MPI_Comm_rank(MPI_COMM_WORLD, &rank)`: Get rank of the process in the communicator `MPI_COMM_WORLD`.
- `MPI_Comm_size(MPI_COMM_WORLD, &size)`: Get the number of processes belonging to the group associated with the communicator `MPI_COMM_WORLD`.

```
#include <mpi.h>

#include <stdio.h>
int main(int argc, char **argv)
{
    int size, rank; char name[256];

    MPI_Init(&argc, &argv); MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    gethostname(name, 256);
    printf("Hello from %d on %s (out of %d procs.!)\\n", rank, name, size);

    MPI_Finalize();
}
```

MPI Messages

A MPI message includes data and metadata (called the envelope).

Metadata

- Process rank (sender and receiver)
- A Communicator (the context of the communication)
- A message tag (can be used to distinguish between messages inside a communicator)

Data

The data are described by the following information:

- Address of the beginning of the buffer
- Number of elements
- Type of the elements

Syntax of send/recv functions

```
int MPI_Send(const void *buf,  
             int count, MPI_Datatype datatype,  
             int dest, int tag, MPI_Comm comm);
```

```
int MPI_Recv(void *buf,  
             int count, MPI_Datatype datatype,  
             int source, int tag, MPI_Comm  
             comm, MPI_Status *status);
```

Elementary datatypes in C

MPI datatype	C datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG DOUBLE	long double
MPI_BYTE	1 Byte

A few more things

The status object

Contains information about the communication (3 fields):

- MPI_SOURCE: the id of the sender.
- MPI_TAG: the tag of the message.
- MPI_ERROR: the error code

The status object has to be allocated by the user.

Read by status. MPI_SOURCE, etc.

Wildcards for receptions

- MPI_ANY_SOURCE: receive from any source
- MPI_ANY_TAG: receive with any tag

Contents

Message Passing Systems

Introduction to MPI

**Point-to-point
communication**

Collective communication

Other features

Blocking communication

`MPI_Send()` and `MPI_Recv()` are blocking communication primitives.

What does blocking means in this context?

Blocking communication

MPL_Send() and MPI_Recv() are blocking communication primitives.

What does blocking means in this context?

- **Blocking send:** When the call returns, it is safe to reuse the buffer containing the data to send.
 - ▶ It does not mean that the data has been transferred to the receiver.
 - ▶ It might only be that a local copy of the data has been made
 - ▶ It may complete before the corresponding receive has been posted
- **Blocking recv:** When the call returns, the received data are available in the buffer for use.

Beware: if recv is not completed, the program stalls (deadlock)

Communication Mode

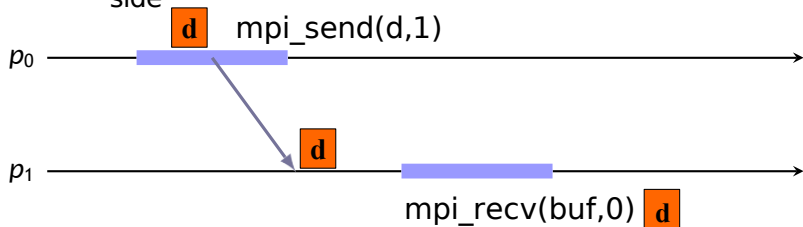
- Standard (`MPI_Send()`)
 - ▶ The send may buffer the message locally or wait until a corresponding reception is posted.
- Buffered (`MPI_BSend()`)
 - ▶ Force buffering if no matching reception has been posted.
- Synchronous (`MPI_SSend()`)
 - ▶ The send cannot complete until a matching receive has been posted (the operation is not local)
- Ready (`MPI_RSend()`)
 - ▶ The operation fails if the corresponding reception has not been posted.
 - ▶ Still, send may complete before reception is complete

Protocols for standard mode

A taste of the implementation

Eager protocol

- Data sent assuming receiver can store it
- The receiver may not have posted the corresponding reception
- This solution is used only for small messages (typically < 64kB)
 - This solution has low synchronization delays
 - It may require an extra message copy on destination side

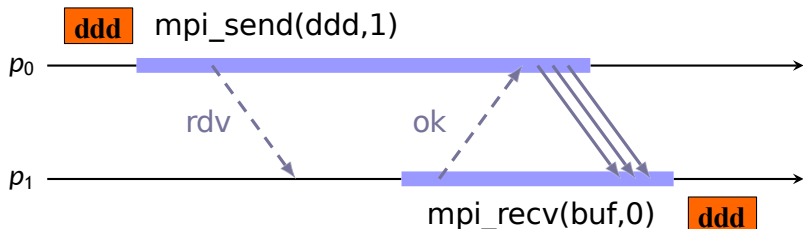


Protocols for standard mode

A taste of the implementation

Rendezvous protocol

- Message is not sent until the receiver is ok
- Protocol used for *large messages*
 - ▶ Higher synchronization cost
 - ▶ If the message is big, it should be buffered on sender side.



Non blocking communication

Basic idea: dividing communication in **two logical steps**

- Posting a request: Informing the library of an operation to be performed
- Checking for completion: Verifying whether the action corresponding to the request is done

Posting a request

- Non-blocking send: `MPI_Isend()`
- Non-blocking recv: `MPI_Irecv()`
- They return a `MPI_Request` to be used to check for completion

Non blocking communication

Checking request completion

- Testing if the request is completed : `MPI_Test()`
 - ▶ Returns true or false depending if the request is completed
- Other versions to test several requests at once (suffix: any, some, all)

Waiting for request completion

- Waiting until the request is completed : `MPI_Wait()`
- Other versions to wait for several requests at once (suffix: any, some, all)

Overlapping communication and computation

Non-blocking communication primitives allow **trying to overlap communication and computation** (performance increase)

- Better performance if the two occur in parallel

```
MPI_Isend(..., req);
```

```
...
```

```
/* run some computation */
```

```
...
```

```
MPI_Wait(req);
```

However, things are not that simple:

- MPI libraries are not multi-threaded (by default)
 - ▶ The only thread is the application thread (no progress thread)
- The only way to get overlapping is through specialized hardware
 - ▶ The network card has to be able to manage the data transfer alone

Match incoming mess. and rec. requests

MPI communication channels are *First-in-First-out* (FIFO)

- Note however that a communication channel is defined in the context of a communicator

Matching rules

- When the reception request is named (source and tag defined), it is matched with the next arriving message from the source with correct tag.
- When the reception request is anonymous (MPI_ANY_SOURCE), it is matched with next message from any process in the communicator
 - ▶ Note that matching is done when the envelope of the message arrives.

Discussion about performance of P2P communication

Things to have in mind to get good communication performance:

- Avoid extra copies of the messages
 - Reception requests should be posted before corresponding send requests
- Reduce synchronization delays
 - Same solution as before
 - The latency of the network also has an impact
- Take into account the topology of the underlying network
 - Contention (competition for resources among different processes or devices) can have a dramatic impact on performance

Contents

Message Passing Systems

Introduction to MPI

Point-to-point
communication

Collective communication

Other features

Collective communication

A collective operation involves all the processes of a communicator.

All the classic operations are defined in MPI:

- Barrier (global synchronization, **to avoid if possible**)
- Broadcast (one-to-all)
- Scatter/ gather
- Allgather (gather + all members receive the result)
- AllToAll
- Reduce, AllReduce (Example of op: sum, max, min)
- etc.

There are v versions of some collectives (Gatherv, Scatterv, Allgatherv, Alltoallv):

- They allow using a vector of send or recv buffers.

Example with broadcast

Syntax

```
int MPI_Bcast( void *buffer,  
               int count, MPI_Datatype datatype,  
               int root, MPI_Comm comm );
```

Broadcast Hello

```
#include <mpi.h>  
int main(int argc, char *argv[])  
{  
    char msg[20]; int my_rank;  
    MPI_Init(&argc, &argv);  
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);  
  
    if (my_rank == 0)  
        strcpy(msg, "Hello from 0!");  
    MPI_Bcast(msg, 20, MPI_CHAR, 0,  
              MPI_COMM_WORLD);  
    printf("rank %d: I received %s\n", my_rank, msg);  
    MPI_Finalize();  
}
```

About collectives and synchronization

What the standard says

A collective communication call **may, or may not**, have the effect of synchronizing all calling processes.

- It cannot be assumed that collectives synchronize processes
 - ▶ Synchronizing here means that no process would complete the collective operation until the last one entered the collective
 - ▶ `MPIBarrier()` still synchronize the processes
- Why is synchronization useful?
 - ▶ Ensure correct message matching when using anonymous receptions
 - ▶ Avoid too many *unexpected* messages (where the reception request is not yet posted)
 - ▶ Debugging!

About collectives and synchronization

What about real life?

- In most libraries, collectives imply a synchronization
 - An implementation without synchronization is costly
- A user program that assumes no synchronization is erroneous

Incorrect code (High risk of deadlock)

```
if(my_rank == 1) MPI_Recv(0);
```

```
MPI_Bcast(...);
```

```
if(my_rank == 0) MPI_Send(1);
```

Implementation of collectives

- MPI libraries implement several algorithms for each collective operation
- Different criteria are used to select the best one for a call, taking into account:
 - ▶ The number of processes involved
 - ▶ The size of the message
- Collective operations are expensive because of contention
- A supercomputer may have its own custom MPI library
 - ▶ Take into account the physical network to optimize collectives

Contents

Message Passing Systems

Introduction to MPI

Point-to-point
communication

Collective communication

Other features

Derived datatypes

We have already introduced the basic datatypes defined by MPI

- They allow sending contiguous blocks of data of one type

Sometimes one will want to:

- Send non-contiguous data (a sub-block of a matrix)
- Buffers containing different datatypes (an integer count, followed by a sequence of real numbers, structures, attributes in an object,...)

One can define derived datatypes

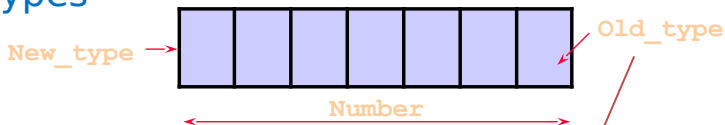
Derived datatypes

- A derived datatype is defined based on a type-map
 - ▶ A type-map is a sequence of pairs {dtype, displacement}
 - ▶ The displacement is an address shift relative to the basic address

Committing types

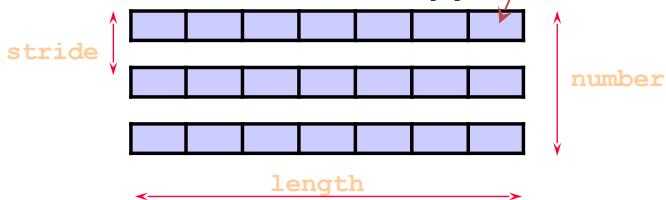
- MPI_Type_commit()
 - ▶ Commits the definition of the new datatype
 - ▶ **A datatype has to be committed before it can be used** in a communication
 - ▶ Mark the datatype object for de-allocation
- MPI_Type_free()

Contiguous and vector datatypes



`MPI_Type_contiguous(number, Old_type, &New_type)`

Vector datatypes



`MPI_Type_vector(number, length, stride, Old_type, &New_type)`

`stride` in number of `Old_types`

Example

```
MPI_Datatype Col_Type, Row_Type; MPI_Comm  
comm;
```

```
MPI_Type_contiguous(6, MPI_REAL, &Col_Type);  
MPI_Type_commit(&Row_Type);
```

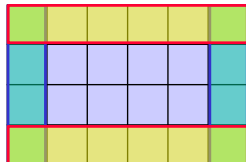


```
MPI_Type_vector(4, 1, 6, MPI_REAL, &Row_Type);  
MPI_Type_commit(&Col_Type);
```

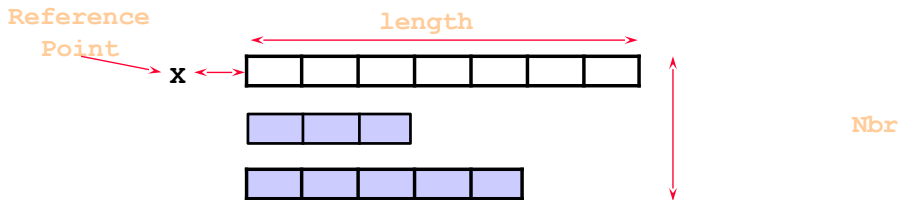


```
...  
MPI_Send(A(0,0), 1, Col_Type, west, 0, comm);  
MPI_Send(A(0,5), 1, Col_Type, east, 0, comm);  
MPI_Send(A(0,0), 1, Row_Type, north, 0, comm);  
MPI_Send(A(3,0), 1, Row_Type, south, 0, comm);  
...
```

```
MPI_Type_free(&Col_Type);  
MPI_Type_free(&Row_Type);
```



Indexed datatype



`MPI_Type_indexed(Nbr, lengths, strides, Old_type, &New_type)`
where `in number` of `Old_types`

- Lengths and strides are 1D arrays of length `Nbr`
- The `New_type` is made of `Nbr` arrays (`i=0; i<Nbr`) each of size `lengths(i)`, each being at `strides(i)` of the `Old_type` (where `(i)` in number of items except for `MPI_Type_hindexed`.)

Performance with derived datatypes

Derived datatypes should be used carefully:

- By default, the data are **copied** into a contiguous buffer being sent (no zero-copy)
- **Special hardware support** is required to avoid this extra copy

Operations on communicators

New communicators can be created by the user:

- Duplicating a communicator (`MPI_Comm_dup()`)
 - ▶ Same group of processes as the original communicator
 - ▶ New communication context

```
int MPI_Comm_dup(MPI_Comm comm, MPI_Comm  
*newcomm);
```

- Splitting a communicator (`MPI_Comm_split()`)

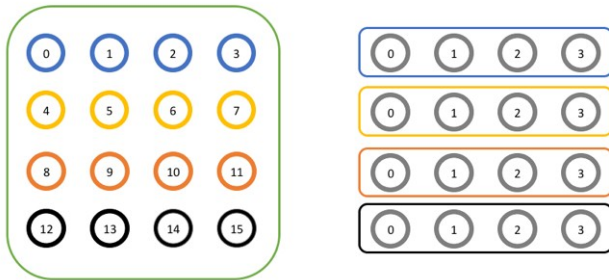
```
int MPI_Comm_split(MPI_Comm comm, int color, int key,  
MPI_Comm *newcomm);
```

 - ▶ Partitions the group associated with `comm` into disjoint subgroups.
 - ▶ Each subgroup contains the PEs that passed the same value of `color`.
 - ▶ Within each subgroup, the processes are ranked in the order defined by the value of the argument `key` (**if equal keys, the library decides**).
 - ▶ Useful when defining hierarchy of computation

MPI_Comm_split

```
int MPI_Comm_split(MPI_Comm comm, int color, int key,  
                  MPI_Comm *newcomm);
```

Split a Large Communicator Into Smaller Communicators



MPI_Comm_split

int MPI_Comm_split(MPI_Comm comm, int color, int key,
MPI_Comm *newcomm);

```
// Get the rank and size in the original communicator
int world_rank, world_size;
MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
MPI_Comm_size(MPI_COMM_WORLD, &world_size);

int color = world_rank / 4; // Determine color based on row

// Split the communicator based on the color and use the
// original rank for ordering
MPI_Comm row_comm;
MPI_Comm_split(MPI_COMM_WORLD, color, world_rank, &row_comm);

int row_rank, row_size;
MPI_Comm_rank(row_comm, &row_rank);
MPI_Comm_size(row_comm, &row_size);

printf("WORLD RANK/SIZE: %d/%d \t ROW RANK/SIZE: %d/%d\n",
       world_rank, world_size, row_rank, row_size);

MPI_Comm_free(&row_comm);
```

Warning

The goal of this presentation is only to provide an overview of the MPI interface.

Many more features are available, including:

- One-sided communication
- Non-blocking collectives
- Process management
- Inter-communicators
- etc.

MPI 3.1 standard is a more-than-1000-page document

References

- Many resources available on the Internet
- The man-pages (usually not very informative)
- The specification documents are available at:
<http://mpi-forum.org/docs/>